

Theory Questions

Function

- **Definition:** A function is a block of reusable code that is defined using the `def` keyword. It can exist independently and does not need to be associated with any object.
- **Call Syntax:** Invoked using the function name, followed by parentheses and arguments (if any).
- **Scope:** Functions can be defined at the global level or within another function, making them independent entities.

Example:

python

Copy code

```
def greet(name):  
    return f"Hello, {name}!"  
  
print(greet("Avanindra"))  
  
# Output: Hello, Avanindra!
```

Definition: A method is similar to a function but is associated with an object or class. It operates on the object it belongs to and often modifies or interacts with the object's state.

Call Syntax: Invoked on an object using the dot (.) operator.

Scope: Methods are defined within a class and have access to the instance (`self`) or class (`cls`) they belong to.

Example:

python

Copy code

```
class Greeter:  
    def __init__(self, name):  
        self.name = name  
  
    def greet(self):  
        return f"Hello, {self.name}!"  
  
greeter = Greeter("Avanindra")  
print(greeter.greet())  
# Output: Hello, Avanindra!
```

Function Arguments and Parameters in Python:

Parameters: Variables listed in the function definition to accept input.

Example:

```
def greet(name): # 'name' is a parameter
    print(f"Hello, {name}!")
```

1.

Arguments: Actual values passed to the function when called.

Example:

```
greet("Avanindra") # "Avanindra" is an argument
```

2.

Types of Arguments:

Positional Arguments: Passed in order.

```
def add(a, b):
    return a + b
print(add(2, 3)) # Output: 5
```

1.

Keyword Arguments: Passed by name.

```
def greet(name, msg):
    print(f"{msg}, {name}!")
greet(name="Avanindra", msg="Good morning") # Output: Good morning, Avanindra!
```

2.

Default Arguments: Provide default values.

```
def greet(name, msg="Hello"):
    print(f"{msg}, {name}!")
greet("Avanindra") # Output: Hello, Avanindra!
```

3.

4. **Variable-Length Arguments:** Accept multiple arguments.

***args:** For non-keyword arguments.

```
def add(*nums):
```

```
    return sum(nums)
print(add(1, 2, 3)) # Output: 6
```

○

****kwargs:** For keyword arguments.

```
def display(**info):
    print(info)
display(name="Avanindra", age=25) # Output: {'name': 'Avanindra', 'age': 25}
```

Ways to Define and Call a Function in Python

1. Simple Function

Definition: Use `def` keyword.

Call: Use the function name with parentheses.

```
def greet():
    print("Hello!")

greet() # Output: Hello!
```

2. Function with Parameters

Definition: Include parameters in parentheses.

Call: Pass arguments.

```
def greet(name):
    print(f"Hello, {name}!")

greet("Avanindra") # Output: Hello, Avanindra!
```

3. Function with Return Value

Definition: Use `return` to send back a value.

Call: Capture or use the returned value.

```
def add(a, b):
```

```
    return a + b

result = add(3, 5)

print(result) # Output: 8
```

-

4. Lambda (Anonymous) Function

Definition: Use lambda for short, single-line functions.

Call: Assign to a variable or use directly.

```
square = lambda x: x * x

print(square(4)) # Output: 16
```

5. Function with Default Parameters

Definition: Provide default values.

Call: Omit or provide arguments.

```
def greet(name="Guest"):

    print(f"Hello, {name}!")

greet()          # Output: Hello, Guest!

greet("Avanindra") # Output: Hello, Avanindra!
```

6. Variable-Length Arguments

Definition: Use ***args** for multiple positional arguments, ****kwargs** for multiple keyword arguments.

Call: Pass as needed.

```
def info(*args, **kwargs):

    print(args, kwargs)

info(1, 2, name="Avanindra", age=25) # Output: (1, 2) {'name': 'Avanindra', 'age': 25}
```

The return statement in a Python function is used to send a value or result back to the caller. It allows the function to output data after performing its operations.

Iterable

- **Definition:** An object that can be iterated over (e.g., lists, tuples, strings, dictionaries).
- **How It Works:** Contains an `__iter__()` method that returns an iterator.

Example:

python

Copy code

```
my_list = [1, 2, 3] # Iterable
```

```
for item in my_list:
```

```
    print(item) # Outputs: 1, 2, 3
```

2. Iterator

- **Definition:** An object representing a stream of data, produced one item at a time.
- **How It Works:**
 - Has an `__iter__()` method (returns itself).
 - Has a `__next__()` method to fetch the next item.
 - Raises `StopIteration` when no more items are available.

A **generator** is a special type of iterator that simplifies the process of creating iterators. Generators allow you to produce a sequence of values lazily (on demand) without storing the entire sequence in memory, which makes them memory-efficient.

Advantages of Generators Over Regular Functions

1. **Memory Efficiency:** Generate values one at a time without storing the entire sequence in memory.
2. **Improved Performance:** Compute values only when needed (lazy evaluation).
3. **Simpler Code:** Easier to write iterators compared to classes with `__iter__()` and `__next__()`.
4. **State Retention:** Maintain state between calls without additional variables.

5. **Supports Infinite Sequences:** Can handle infinite data streams without memory issues.
6. **Pipelining:** Enable chaining for efficient and modular data processing.

Lambda Function in Python

A **lambda function** is an anonymous, single-expression function defined using the `lambda` keyword. It is compact and often used for short, simple operations.

When to Use Lambda Functions

1. **Short and Simple Operations:** For one-line functions without the need for a full `def` block.
2. **Inline Use:** When a function is needed temporarily, such as in `map()`, `filter()`, or `sorted()`.
3. **Anonymous Use:** When naming a function is unnecessary.

The `map()` function applies a given function to all items in an iterable (like a list, tuple, etc.) and returns an iterator that yields the results.

- **Apply a Function to All Items:** It allows you to process each item in an iterable without using a loop.
- **Returns an Iterator:** The result is an iterator, so you often need to convert it to a list or another collection.

`map()`:

- **Purpose:** Applies a given function to each item of an iterable and returns an iterator of the results.
- **Output:** A new iterable where each item is the result of applying the function to the original iterable's items.
- **Common Use:** Transformation of data (e.g., squaring numbers, converting strings to uppercase).

Example:

python

Copy code

```
numbers = [1, 2, 3]
result = map(lambda x: x * 2, numbers)
print(list(result)) # Output: [2, 4, 6]
```

-

`reduce()` (from `functools`):

- **Purpose:** Applies a function cumulatively to the items of an iterable, reducing the iterable to a single value.
- **Output:** A single value that is the result of combining the elements of the iterable.
- **Common Use:** Accumulation or aggregation (e.g., summing numbers, finding the product of numbers).

Example:

python

Copy code

```
from functools import reduce
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, numbers)
print(result)
```

•

filter():

- **Purpose:** Filters the elements of an iterable based on a function that returns True or False.
- **Output:** An iterable containing only those elements for which the function returns True.
- **Common Use:** Filtering elements from a collection (e.g., selecting even numbers, filtering out negative values).

Example:

python

Copy code

```
numbers = [1, 2, 3, 4]
result = filter(lambda x: x % 2 == 0, numbers)
```

- `print(list(result))` # Output: [2, 4]

Step by Step Breakdown

from code:

$[47, 11, 42, 13]$

$x = 47 \quad y = 11$

from function `import reduce`
 $numbers = [47, 11, 42, 13]$

`result = reduce(lambda x, y: x+y,`

`numbers)`
`print (result)`

→ Reduce is Applied sequentially

$x = 47, y = 11$

1st Iter.

$47 + 11 = 58, \text{ result} = 58$

2nd Iter.

$x = 58, y = 42$

$58 + 42 = 100, \text{ result} = 100$

3rd Iter.

$x = 100, y = 13$

$100 + 13 = 113$

Final result = 113

After all its, result = 113